



MODULE 2: PRESENTATION

CSS Foundations: Rules That Make Structure Visible

Write valid style rules, target elements reliably, and reason about the box model.

CSS is a language of rules

Cascading Style Sheets (CSS) applies visual formatting to HTML documents. Every CSS instruction follows a strict, predictable grammar.

```
h1 {  
  color: #4d176e;  
  font-size: 48px;  
}
```



Selector

Finds the target HTML elements on the page. In this case, it finds all `<h1>` tags.



Property

The specific visual characteristic you want to change (e.g., color, spacing, font).



Value

The chosen setting for that property. Together, a property and value form a **declaration**.

Link an external stylesheet

Keep content (HTML) and presentation (CSS) separate so the browser can find and apply styles.

INDEX.HTML

```
<!DOCTYPE html> <html lang="en"> <head> <meta
charset="UTF-8"/> <title>Course Page</title>
<link rel="stylesheet" href="styles.css"/>
</head>

<body> <h1>Software Engineering</h1> </body>
</html>
```

SEPARATION OF CONCERNS

Keeping CSS in a separate .css file lets many pages share the same design; changing a color updates every linked page.

THE <LINK> ELEMENT

Placed in the HTML <head>, the <link rel="stylesheet"> tells the browser where to download styles.

Type, class, and ID selectors

Selectors determine which HTML elements receive your styles. Choose the right selector based on how broadly you want the rule to apply.

Type Selector

Targets all elements of a specific HTML tag. Best for setting broad defaults like typography and margins.

```
h2 {  
  color: #1f0d2e;  
}
```

```
<h2>Course Info</h2>
```

SPECIFICITY: LOW (0,0,1)

Class Selector

Targets elements with a specific class attribute. Reusable and safe. This is the workhorse of modern CSS.

```
.btn-primary {  
  background: #f47945;  
}
```

```
<a class="btn-primary">Apply</a>
```

SPECIFICITY: MEDIUM (0,1,0)

ID Selector

Targets a single, unique element on the page. Use sparingly, as its high specificity makes it hard to override.

```
#hero-banner {  
  height: 400px;  
}
```

```
<header id="hero-banner">...  
</header>
```

SPECIFICITY: HIGH (1,0,0)

Combinators express relationships

Target elements based on their position in the HTML document tree. Combinators keep your markup clean by reducing the need for extra classes.

```
/* Descendant: space */
article p { color: #333; }

/* Child: > */
ul > li { margin-bottom: 8px; }

/* Adjacent Sibling: + */
h2 + p { margin-top: 0; }
```



Descendant

Selects elements that are inside the first element, no matter how deeply nested they are.



Child

Selects elements that are **direct** children of the first element. It will not select grandchildren.



Adjacent Sibling

Selects an element that comes **immediately after** the first element, sharing the same parent.

Pseudo-classes expose meaningful states

Elements change state based on user interaction. Pseudo-classes allow you to style these specific moments without adding JavaScript.

:hover

Applies when a user points a pointing device (like a mouse) at an element. It provides immediate visual feedback that an item is interactive.

```
button:hover {  
  background: #f47945;  
}
```

:focus-visible

Applies when an element receives focus via keyboard navigation. This is critical for accessibility, ensuring keyboard users know where they are.

```
a:focus-visible {  
  outline: 3px solid  
  #f47945;  
}
```

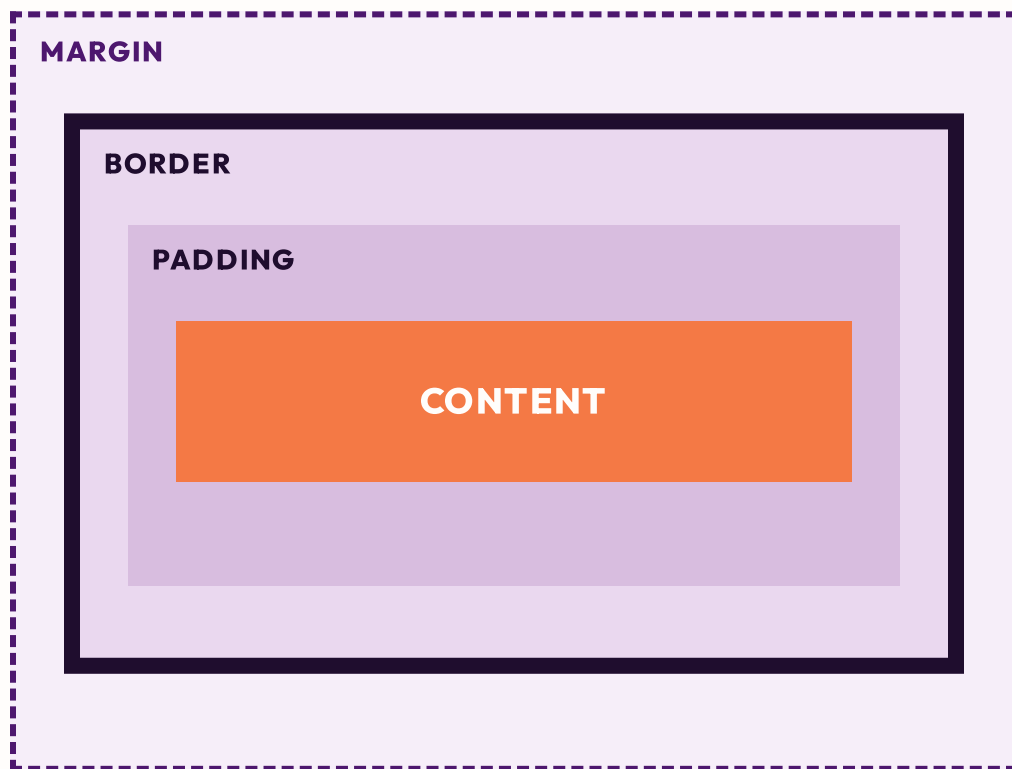
:active

Applies while an element is being activated (e.g., during the exact moment a button is being clicked or tapped).

```
button:active {  
  transform: scale(0.98);  
}
```

The box model explains every element

Every HTML element is a rectangular box. CSS determines the size, spacing, and boundaries of these boxes through four distinct layers.



CONTENT

The actual content of the box, where text and images appear. Sized by width and height properties.

PADDING

Transparent space inside the border. It protects the content and takes on the element's background color.

BORDER

A line that goes around the padding and content. It defines the visible edge of the element's box.

MARGIN

Transparent space outside the border. It pushes other elements away and creates layout separation.

Use the border-box reset

By default, padding and borders add to an element's specified width. The border-box reset changes this behavior so widths remain predictable.

```
/* The Universal Reset */  
html { box-sizing: border-box; }  
*, *::before, *::after { box-  
sizing: inherit; }
```

DEFAULT: CONTENT-BOX

If you set width to 300px, then add 20px padding and a 5px border, the browser adds them together. The box grows larger than you asked.

$$300 + (20 \times 2) + (5 \times 2)$$

= 350px total width

RESET: BORDER- BOX

If you set width to 300px, the browser absorbs the padding and border into that width. The content area shrinks, but the box stays exactly 300px.

300px total width
(content shrinks to 250px)

Spacing has intent

Don't just add space until it looks right. Choose between padding and margin based on the structural job the space needs to perform.



Padding = Protection

Padding is space **inside** the element. It protects the content from hitting the edges of its container.



Margin = Separation

Margin is space **outside** the element. It pushes other elements away to create layout structure and rhythm.

Units are design choices

Hard-coded pixels make designs brittle. Use relative units so UIs adapt to users and screens.

TYPOGRAPHY & SPACING

rem (Root EM)

Scales with the root font size — ideal for consistent type sizing.

em

Relative to the element's font size — great for component padding.

px (Pixels)

Absolute units for fine details like borders and hairlines.

LAYOUT & CONTAINERS

% (Percentages)

Fluid sizing relative to a parent container's dimensions.

vh / vw (Viewport)

Relative to the viewport size — useful for full-screen sections.

fr (Fractional)

Grid fractions distribute available space (e.g. 1fr 2fr).

Inspect before guessing

Use DevTools to see which rules apply and which are overridden.

THE ELEMENTS PANEL

Shows the live DOM tree — select an element to inspect its structure.

THE STYLES PANE

All rules for the selected element, ordered by specificity.
Overridden props are crossed out.

THE COMPUTED PANE

Resolved pixel values and box-model measurements for the selected element.

STYLES COMPUTED LAYOUT

```
.btn-primary styles.css:42
  background-color: #f47945;
  color: #ffdfdb;
}
```

Lab: Style a programme introduction

Apply CSS foundations to the HTML document you built previously. Transform raw markup into a readable, interactive LEA Labs programme page.

DELIVERABLES

1. The Reset

Link your stylesheet and apply the universal border-box reset to ensure predictable sizing.

2. Typography & Color

Use type selectors to set the base font family, line-height, and background colors.

3. Spacing

Use class selectors to apply intentional margin (for separation) and padding (for internal space).

4. Interactive States

Add :hover and :focus-visible pseudo-classes to all links and buttons.

SUCCESS CRITERIA

- The HTML file remains unchanged; all styling is external.
- No inline styles or deprecated HTML presentation attributes are used.
- Text is readable with appropriate contrast and line-height.
- Keyboard navigation clearly highlights focused elements.

EXTENSION CHALLENGE

Use combinators (like the adjacent sibling selector +) to adjust the margin between adjacent paragraphs, without adding any new classes to the HTML.